

GROUP 04

Student Attendance System Specification

I. Student Attendance System	1
II. Functional Requirements of the Student Attendance System	4
III. Non-Functional Requirements	14
IV. Data Flow Diagram	17
V. Use case Diagram	23
VI. Class Diagram	38
VII. Data Model	39
VIII. Interface Design Description	39

I. Student Attendance System

1. IMPORTSNT NOTES:

-) Data about lecturers, students, and classes is provided by the university.
-) The system only reads and uses this data; it does NOT create or edit lecturer, student, or class information.
-) Class codes, student IDs, and emails are assigned by the university and are unique.
-) The system only checks data validity (correct format, no missing data) before loading the data.
-) The system does not handle course registration and does not manage adding, editing, or deleting students or lecturers. The data is preloaded from CSV files.

2.Initial Data of the System.

2.1. Lecturer and class data

The system has a preloaded list of lecturers who are assigned to classes.

Each lecturer includes the following information:

- Lecturer name
- University email (format: name without accents + @ut.edu.vn)
- Citizen ID number (used as the login password)
- List of classes being taught

Each class includes:

- Class ID (8 digits, for example: 10100234)

Class ID structure:

- First 3 digits: Training program code (101 = High Quality Program, 102 = IT, ...)
- Middle 2 digits: Fixed as 00
- Last 3 digits: Class identifier

Examples: 10100234, 10200123, 10300456

Note: Each class has a unique ID with no duplicates. Class IDs are assigned by the university and are preloaded into the system.

2. 2.Student data

The system has a preloaded list of students. Each student includes the following information:

- Student name
- University email (format: name without accents + @ut.edu.vn)
- Student ID
- List of registered classes (including course name and class ID)

This data is loaded into the system and is used for login, class management, and attendance.

Note: Each student has a unique student ID with no duplicates. The ID is assigned by the university and is preloaded into the system.

2.3. Initial CSV file structure

2.3.1. Lecturer file (lecturers.csv):

+) **Structure:** TenGiangVien;Email;SoCCCD;DanhSachLopHoc

Example: Nguyen Van

[Hai;nguyenvanhai@ut.edu.vn;001234567890;10100234,10100235](mailto:nguyenvanhai@ut.edu.vn)

+) **Note:** *DanhSachLopHoc* is separated by commas, with no spaces.

2.3.2. Student file (students.csv):

+)Structure: TenSinhVien;Email;MaSinhVien;DanhSachLopHoc

+)Example: Nguyen Van An;nguyenvanan@ut.edu.vn;NVA1234;10100234,10100123
+)Note: *DanhSachLopHoc* is separated by commas, with no spaces

2.3.3.Class file (classes.csv):

+)Structure: MaLop;TenMonHoc;EmailGiangVien;DanhSachSinhVien

+)Example: 10100234;Lap trinh Java;nguyenvanhai@ut.edu.vn;NVA1234,TTB5678,LMT9012

+)Note: *DanhSachSinhVien* is separated by commas, with no spaces

2.3.4.Class schedule file (class_schedule.csv):

MaLop;NgayHoc;GioBatDau;GioKetThuc

Example:

10100234;15/01/2026;08:00;09:30

10100234;22/01/2026;08:00;09:30

Note: This data is loaded into the system.

3. DATA GENERATED AND MANAGED BY THE SYSTEM.

Note: This data is generated and managed by the system for processing and storing attendance. It is different from the data provided by the school (such as *lecturers.csv*, *students.csv*, *classes.csv*, *class_schedule.csv*).

Each class has **2 separate CSV files** to store attendance information:

3.1.Student list file of the class (class_[MaLop]_students.csv):

Structure: hotensinhvien;mssv;sobuoihoc **Example**

file: class_10100234_students.csv

Example:

Nguyen Van An;NVA1234;8

Tran Thi Binh;TTB5678;7

Le Minh Tuan;LMT9012;9

Meaning: *sobuoihoc* is the total number of sessions that the student has successfully attended in this course.

The file **class_[MaLop]_students.csv** is a system-generated data file. It is created from **classes.csv** when the class is initialized in the system, and it is allowed to be updated.

3.2. Class attendance session file (class_[MaLop]_sessions.csv):

Structure: malop;batdau;ketthuc;matkhau;svcomat

Example:

10100234;17/01/2026 08:00;17/01/2026 09:30;;NVA1234,TTB5678,LMT9012

Meaning:

- **malop:** Class ID
- **batdau:** Attendance session start time in the format DD/MM/YYYY HH:MM
- **ketthuc:** Attendance session end time in the format DD/MM/YYYY HH:MM
- **matkhau:** Attendance session password (leave empty if not used)
- **svcomat:** List of student IDs who attended, separated by commas

Note: The file **classes.csv** is considered the main data source to determine the student list of a class.

The field **DanhSachLopHoc** in **students.csv** is only used to quickly display the student's class list after login.

4. Rules for determining attendance status:

- +) The system does NOT store the “Absent” status directly.
- +) A student is considered “**Present**” when the student ID **appears** in the *svcomat* list of that session.
- +) A student is considered “**Absent**” when the student ID **does NOT appear** in the *svcomat* list of that session.

II. Functional Requirements of the Student Attendance System

1. LOGIN

1.1. Student login

- Students must log in before accessing the system.
- The username is the university email provided by the school (ending with @ut.edu.vn), and the password is the student ID.
- The student ID is created from the first letters of the full name combined with the last 4 digits of the citizen ID number.
- For example, a student named Nguyen Van An with a citizen ID ending in 1234 will have the student ID **NVA1234**.
- Each time a student wants to access the system, they must log in again. The system does not store login sessions.
- If the student enters an incorrect email or password, the system will display the message: **"Email or password is incorrect. Please try again."**
- If the student logs in successfully, the system will display the message: **"System access successful."**

1. 2.Lecturer login

- Lecturers must also log in to access the system.
- The username is the university email (ending with @ut.edu.vn), and the password is the lecturer's citizen ID number.
- Similar to students, each time lecturers access the system, they must log in again from the beginning.
- If a lecturer enters an incorrect email or password, the system will display the message: **"Email or password is incorrect. Please try again."**
- If the lecturer logs in successfully, the system will display the message: **"System access successful."**

2. LOGOUT

2.1.Student logout

- After logging in and using the system, students can use the logout function to exit the system.
- When a student selects the logout function, the system ends the current session and returns the student to the login screen.
- Because the system does not store login sessions, after logging out, if the student wants to continue using the system, they must log in again from the beginning using the university email and the corresponding password.
- After a successful logout, the system will display the message: **"Logout successful."**

2.2.Lecturer logout

- After logging in and using the system functions, lecturers can log out to exit the system.
- When a lecturer selects the logout function, the system ends the current session and returns to the login screen.

- Similar to students, because the system does not store login sessions, each time a lecturer wants to access the system again, they must log in again from the beginning using the university email and password.
- After a successful logout, the system will display the message: **"Logout successful."**

3.VIEW AND SEARCH CLASSES

3.1.Lecturer

- After logging in, lecturers can view the list of all classes they are teaching, together with the fixed class session time for each class on specific dates.
- Class session times are taken from the schedule file (**class_schedule.csv**) and include only classes where **EmailGiangVien** matches the email of the logged-in lecturer.
- Lecturers can enter a specific date (format **DD/MM/YYYY**) to view the list of classes they teach on that day.
- The system filters data based on **class_schedule.csv**, combined with lecturer information, and only displays classes that meet both conditions:

+) Have a class schedule on the entered date

+) Belong to the teaching assignments of the logged-in lecturer

- Lecturers can also search for classes by entering a course name or a class ID.

+) **Search by course name:** the system is case-insensitive and displays all classes with matching course names. The lecturer chooses the specific class.

+) **Search by class ID:** the system returns exactly one class, because the class ID is unique and unambiguous.

- Search results by course name only include classes that the logged-in user (lecturer or student) has access to.
- After filtering by date or searching, the system displays the list of classes that match the filter or search conditions.
- For each displayed class, the system provides the following information:

Course name (from **classes.csv**)

Class ID

Lecturer name

Class date, start time, and end time (from **class_schedule.csv**)

3. 2.Student

- After logging in, students can view the list of all classes they have registered for. The system automatically displays the class list based on the student's login information.
- Students can also search for classes by entering a course name or a class ID. The class ID is assigned by the university.
- After the student searches, the system displays the list of classes that match the filter or search conditions.
- For each displayed class, the system provides the following information:

Course name (from **classes.csv**)

Class ID

Lecturer name

Class date, start time, and end time (from **class_schedule.csv**)

4. VIEW STUDENT LIST IN A CLASS

- After a lecturer selects a class from the list of classes they are teaching, the system allows the lecturer to view the list of students registered in the selected class.
- The student list is taken from the file **class_[MaLop]_students.csv**, corresponding to the selected class ID.
- For each student, the system displays the following information:

Student full name

Student ID

Attendance status of the student in the current class session

5. CREATE ATTENDANCE SESSION

- After a lecturer selects a class, the lecturer can create an attendance session. The lecturer enters the start time (hour, minute, day, month, year) and the end time (hour, minute, day, month, year) during which students are allowed to check in.
 - In addition, the lecturer can set a password for the attendance session if desired. The attendance password must have at least 4 characters and consist of 4 digits from 0 to 9. If a password is set, students must enter the correct password when checking in.
- Note:** The minimum attendance duration must be 5 minutes.
- A lecturer can create multiple attendance sessions for a class, but at any given time, only one active session is allowed.
- After the end time, the system automatically locks the attendance session. Students cannot check in after the session is locked.

Storage process when creating a session:

When a lecturer creates a new attendance session, the system adds a new row to the class attendance session file (**class_[MaLop]_sessions.csv**) with the following information: **malop, batdau, ketthuc, matkhau** (if any), and leaves the **svcomat** field empty (because no student has checked in yet).

6. ATTENDANCE NOTIFICATION

- When a lecturer finishes creating an attendance session, the system displays a notification in that class.
- When a student accesses the class, the student will see a notification that an attendance session is open and how much time remains before it ends.
- If a student accesses the class after the attendance session has ended, the system will display the message:
"Attendance has ended."

7. ATTENDANCE.

Method 1: Students take attendance by themselves

During an open attendance session, students can take attendance for themselves.

Student attendance process:

Students follow these steps:

-) If the lecturer sets a password, the student must enter the correct password first. If the password is wrong, the system shows an error message and allows the student to try again.
-) Then the student enters the word **“present”** (not case-sensitive) to confirm that they are present.
-) The system records the attendance and adds 1 to the student’s attended sessions.

Each student can take attendance only once per session. If a student tries to take attendance again, the system shows the message **“You have already taken attendance for this session.”** If the attendance time is over, the student cannot take attendance anymore.

Processing when a student takes attendance:

When a student successfully takes attendance in an open session:

-) The system adds the student ID to the **svcomat** list of that session in the attendance file **class_[ClassCode]_sessions.csv**.
-) At the same time, the system adds 1 to **sobuoihoc** of that student in the student list file **class_[ClassCode]_students.csv**.

Example:

Student **NVA1234** takes attendance in the session on **15/01/2026**.

Before attendance, the session row in the sessions file has an empty **svcomat**.

After attendance, **svcomat** becomes “**NVA1234**”.

In the students file, **sobuoihoc** of student **NVA1234** increases from **7** to **8**.

Note:

If a student does not take attendance within the allowed time, the system automatically marks the student as “**Absent**” when the session ends. This means the student ID is not in the **svcomat** list and **sobuoihoc** is not increased for that session.

*Attendance data is recorded in **class_[ClassCode]_sessions.csv** and the number of attended sessions (**sobuoihoc**) is updated in **class_[ClassCode]_students.csv**.*

Method 2: Lecturer takes attendance for students (manual confirmation)

When the lecturer wants to control attendance, the system allows the lecturer to take attendance for students instead of students taking attendance by themselves.

Lecturer attendance process:

-) The lecturer opens the attendance session that is currently running for the class.
-) The system shows the list of students of the class, loaded from the file **class_[MaLop]_students.csv**. This is data generated and managed by the system.
-) For each student that the lecturer confirms as present in class, the lecturer enters the word “**có**” to confirm attendance for that student.
-) Each student can be marked only once in each session. If the lecturer tries to mark attendance again for a student who is already recorded, the system shows a related message.
-) When the attendance session ends, students who are not confirmed by the lecturer are considered absent.

Processing when the lecturer marks attendance successfully:

-) The system adds the student ID to the **svcomat** list of that session in the file **class_[MaLop]_sessions.csv**.
-) At the same time, the system adds 1 to **sobuoihoc** of that student in the file **class_[MaLop]_students.csv**.

*Attendance data is recorded in **class_[MaLop]_sessions.csv** and the number of attended sessions (**sobuoihoc**) of students is updated in **class_[MaLop]_students.csv**.*

8. CHECK AND EDIT ATTENDANCE

8.1. Review the attendance results of students

- After an attendance session is closed (attendance time has ended), the lecturer can review the attendance results of students in that session.
- The system displays the list of all students in the class (taken from **class_[MaLop]_students.csv**).
- For each student, the system determines and displays the attendance status in the session according to the following rules:

How the system determines and displays the status:

- +) The system reads the **svcomat** list of the corresponding session in the file **class_[MaLop]_sessions.csv**.
- +) If the student ID **appears** in the **svcomat** list → display status “**Present**”.
- +) If the student ID **does NOT appear** in the **svcomat** list → display status “**Absent**”.
- +) The total number of attended sessions is taken from the **sobuoihoc** field in the file **class_[MaLop]_students.csv**.
- +) The total number of closed sessions is determined by counting the number of rows in the file **class_[MaLop]_sessions.csv** where the end time is earlier than the current time.

Display examples:

Nguyen Van An – Present – Total: 8/10 sessions

Tran Thi Binh – Absent – Total: 7/10 sessions

Each time a student successfully checks in during an active session, the system adds **1** to the student’s total attended sessions (**sobuoihoc**).

8.2. Manual editing of attendance status

After the attendance session is closed, the lecturer can manually edit the attendance status of a student if needed (for example, the student was present but forgot to check in).

Processing when the lecturer edits the status:

Case 1: Change status from “Absent” to “Present”

- The system adds the student ID to the **svcomat** list of the corresponding session in **class_[MaLop]_sessions.csv**.
- At the same time, the system adds **1** to the **sobuoihoc** field of the student in **class_[MaLop]_students.csv**.

Case 2: Change status from “Present” to “Absent”

- The system removes the student ID from the **svcomat** list of the corresponding session in **class_[MaLop]_sessions.csv**.
- At the same time, the system subtracts **1** from the **sobuoihoc** field of the student in **class_[MaLop]_students.csv**.

After editing, the attendance data of the session and the total number of attended sessions of the student are updated in the corresponding CSV files.

9.VIEW ATTENDANCE HISTORY

9.1. Student

- Students can view their own attendance history for each class.
After logging in and selecting a class, students can see the list of all attendance sessions that have taken place in that class.
- For each attendance session, the system displays the following information:

Attendance session time (date, start time, and end time)

Student’s status in that session: “**Present**” or “**Absent**”

Total attended sessions / total closed sessions

-How the system determines the status for history:

- The system reads all attendance sessions in the class session file
- For each session, the system checks whether the student ID appears in the **svcomat** list
- If it **appears** → status “**Present**”
- If it **does NOT appear** → status “**Absent**”

Example: Attendance history for Java Programming (Class 10100234):

Session 1: 15/01/2026, 08:00–09:30: Present

Session 2: 17/01/2026, 08:00–09:30: Absent

Total: 1/2 sessions attended.

9.2. Lecturer

- Lecturers can view the history of all attendance sessions they have created for each class. After logging in and selecting a class, lecturers can see the list of attendance sessions created for that class.
- For each attendance session, the system displays the following information:

Attendance session time (date, start time, and end time)

Session status: **“Open”** or **“Closed”**

Number of students who checked in / total number of students in the class

Whether a password is required

Lecturers can choose to view the details of a specific attendance session to see the list of students who have checked in and those who have not.

Example:

Course: Java Programming

Class: 10100234

Session 1: 15/01/2026, 08:00–09:30: Closed – 32/35 students present

Session 2: 17/01/2026, 08:00–09:30: Closed – 30/35 students present

Session 3: 19/01/2026, 08:00–09:30: Open – 12/35 students checked in

Total: 2/3 sessions closed.

10. DELETE ATTENDANCE SESSION

The lecturer can delete any attendance session that has been created, no matter whether the session is **open or closed**, and whether students have checked in or not.

-Steps to delete an attendance session:-

+)The lecturer selects a class and views the list of attendance sessions.

- +)The lecturer selects the attendance session to delete.
- +)Before deleting, the system shows a confirmation message:
"Are you sure you want to delete this attendance session? All attendance data will be deleted."
- +)The lecturer can confirm or cancel: If confirmed, the system deletes the session and shows:
"Attendance session deleted successfully." If canceled, the system does not delete the session.

-Processing when deleting a session:

When the lecturer confirms deleting a closed attendance session:

- +)The system reads the **svcomat** list of that session to know which students have checked in.
- +)For each student whose student ID is in the **svcomat** list:
 -)The system decreases the **sobuoihoc** value of that student by **1** in the student list file.
- +)The system deletes the attendance session line from the session file.

After deletion, the system updates the attendance session CSV file of the class.

Note: When deleting a closed attendance session, the total number of attended sessions and the total number of closed sessions must be updated correctly. The attended sessions that students have checked in during that session will be subtracted from their total attended sessions out of the total closed sessions.

Example:

A student has **8/10 sessions**.

After the lecturer deletes **one session** that the student attended, the result is **7/9 sessions** (both the number of attended sessions and total sessions decrease by 1).

When a lecturer confirms deleting an open session:

- +) Delete the corresponding session line from the attendance session file (class_[MaLop]_sessions.csv).
- +) Do not change the sobuoihoc (total attended sessions) of any student.

- +) Do not change the total number of closed sessions.

III. Non-Functional Requirements

1.Product requirement

1.1.Efficiency Requirements (Performance requirements)

- Response time (Response time):
 - +) System login time must not be more than 3 seconds
 - +) Class list loading time must not be more than 2 seconds
 - +) Attendance action must be processed and responded within 2 seconds
 - +) Class search time must not be more than 1 second
- **Processing capacity:**
 - +) The system must handle at least 20 students taking attendance at the same time in one session without being slow
 - +) The system must support at least 10 lecturers using at the same time

1.2.Reliability Requirements (Reliability requirements):

- **Data accuracy (Accuracy):**
 - +) Attendance data must be 100% accurate, no difference
 - +) Total number of present sessions must be calculated exactly as an integer, no error
 - +) Status “Present” or “Absent” must be recorded correctly
- **Data consistency:**
 - +) When a student takes attendance successfully, both files `class_[MaLop]students.csv` and `class[MaLop]sessions.csv` must be updated at the same time +) When a lecturer deletes an attendance session, data in both files `class[MaLop]students.csv` and `class[MaLop]_sessions.csv` must be updated consistently (the total sessions decreases in the same way)
 - +) There must be no conflict between data in different files
- **Recoverability (Recoverability):**
 - +) The system must be able to recover data when an error happens while writing files +) If there is an error when updating the CSV file, the system must roll back to the previous state
 - +) It is recommended to have a regular data backup (about once a week) -
- **Availability (Availability):**
 - +) The system must be available 24/7 during the semester
 - +) Maintenance downtime must not be more than 2 hours/month

1.3.Usability Requirements (Usability requirements)

- **Learnability (Learnability):**

- +) New users must be able to use the system after at most 2 instructions
- +) The interface must be intuitive, no need to read long documents

-Ease of use (Ease of use):

- +) Main actions (login, attendance, view history) must not need more than 10 steps
- +) Error messages must be clear and easy to understand, not unclear in meaning
- +) The menu/interface must be organized logically and easy to navigate

-Error prevention (Error prevention):

- +) The system must ask for confirmation before important actions (like deleting an attendance session)
- +) Must check input data validity (email, password, time)
- +) Show a warning when the user enters data in the wrong format

1.4.Security Requirements (Security requirements)

-Personal data protection:

- +) The system must protect personal information of each student and lecturer
- +) Sensitive data (CCCD, password) must be encrypted when stored

-Authentication & Authorization (Authentication & Authorization) Each user must log in before using the system

Students can only:

- +) View the list of registered classes
- +) Take attendance in their classes +)
View their own attendance history

Lecturers can only:

- +) View the list of classes they teach
- +) Create/delete attendance sessions for their class
- +) View/edit attendance in their class

Do not store login sessions, users must log in again each time they access

-Protection against unauthorized access:

- +) Login email must have the format @ut.edu.vn
- +) Password must match the data in the system
- +) Attendance session password must have at least 4 digits

-Data integrity (Data integrity):

- +) It is not allowed to edit data directly in the CSV file outside the system
- +) The system must check data validity before writing to the file

1. 5.Dependability Requirements (Overall dependability requirements)

-Fault tolerance (Fault tolerance)

- +) Must handle exception cases (file not found, data has wrong format)
- +) Show friendly error messages instead of stopping the program

-Predictability (Predictability)

- +) The system must work consistently: the same action must give the same result
- +) There must be no random or unpredictable behavior

2. ORGANISATIONAL REQUIREMENTS (Organisational requirements)**2.1.Environmental Requirements (Environment requirements):*****-Development environment:***

- +) Programming language: C++
- +) Required IDE: VS Code
- +) Operating system: Windows

-Deployment environment:

- +) The system must run smoothly on a computer with minimum specs: RAM 4GB, 2-core CPU
- +) CSV file size must not be more than 10MB for each file
- +) The system must run on a personal computer with a C++ compiler installed
- +) Internet connection is required to run the system

Requirements (Operation requirements)***-User group:***

Users must be students or lecturers of the University of Transport and Communications

Students must have:

- +) Student ID given by the university (format: 3 uppercase letters + 4 digits)
- +) Email in the format @ut.edu.vn

Lecturers must have:

- +) CCCD number (12 digits)
- +) Email in the format @ut.edu.vn given by the university

Use process:

- +) Users must log in each time they use it
- +) Accounts must not be shared between many people
- +) Must follow the system's business rules

2.3. Development Requirements (Development requirements)***Development process:***

- +) The system must be developed using Agile/Scrum process

2.4. Regulatory Requirements (Compliance requirements)

-Follow university rules:

- +) Email format must follow the university standard: @ut.edu.vn
- +) Student ID format must follow the university rules
- +) Class code is provided by the university and each class has a unique code

-Data rules:

- +) The system is NOT allowed to add/edit/delete student or lecturer information
- +) Only allowed to load data from the original CSV file provided by the university
- +) Only allowed to update attendance-related data

3. EXTERNAL REQUIREMENTS (External requirements)

3.1. Legislative Requirements (Legal requirements)

-Personal data protection:

- +) The system must follow Vietnam's personal data protection law (if any)
- +) Must not share, sell, or disclose personal information to a third party
- +) Must inform users about how data is collected and used

-Privacy:

- +) Students can only view their own attendance data
- +) Lecturers can only view data of the class they teach
- +) It is not allowed to access other people's data

3.2. Interoperability Requirements (Interoperability requirements)

-Data format:

- +) The system must use CSV files as the database
- +) CSV files must follow the defined structure exactly

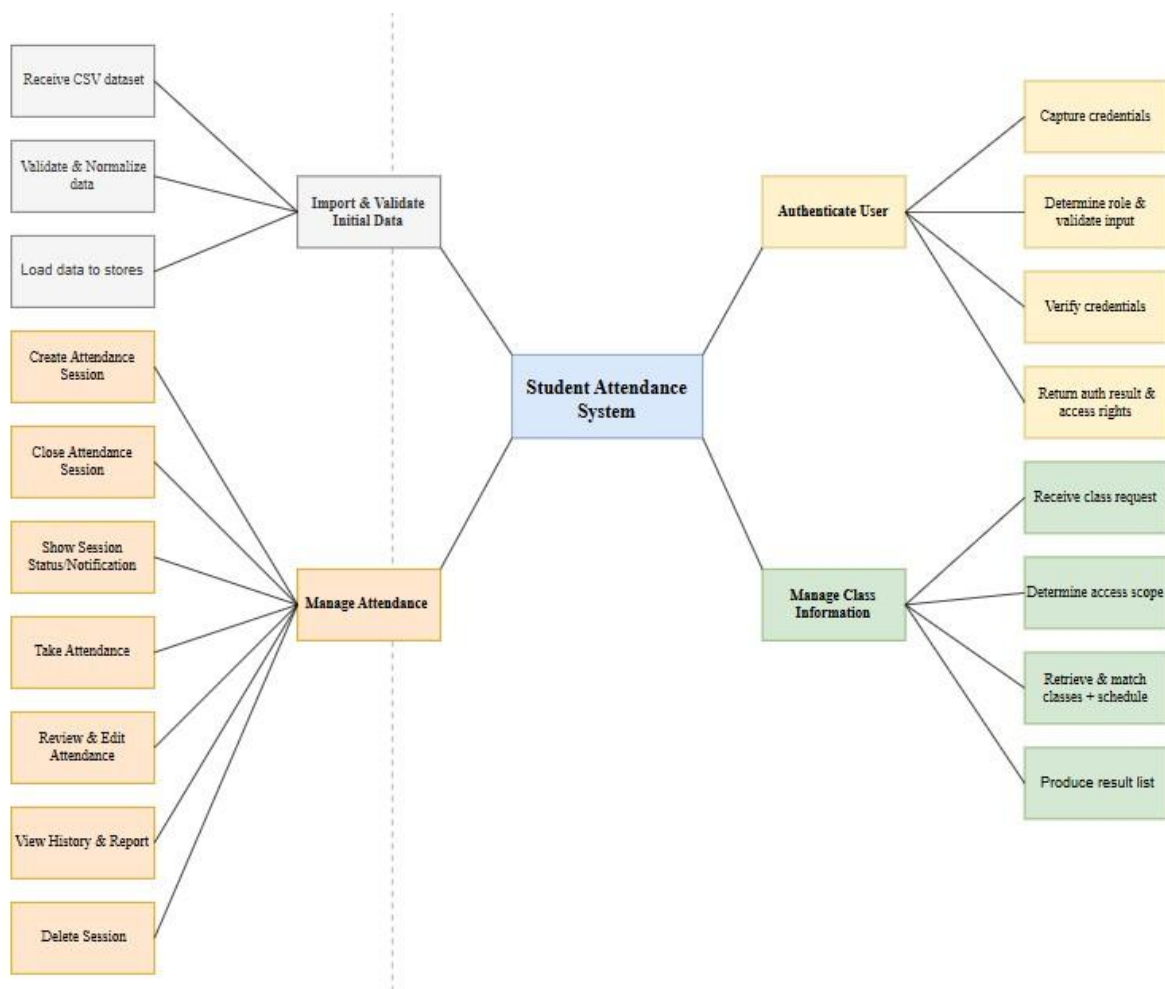
3.3. Ethical Requirements (Ethical requirements)

-Responsibility:

- +) Attendance data must be accurate and reliable
- +) Must not fake or manipulate attendance data
- +) Must be responsible for protecting user data

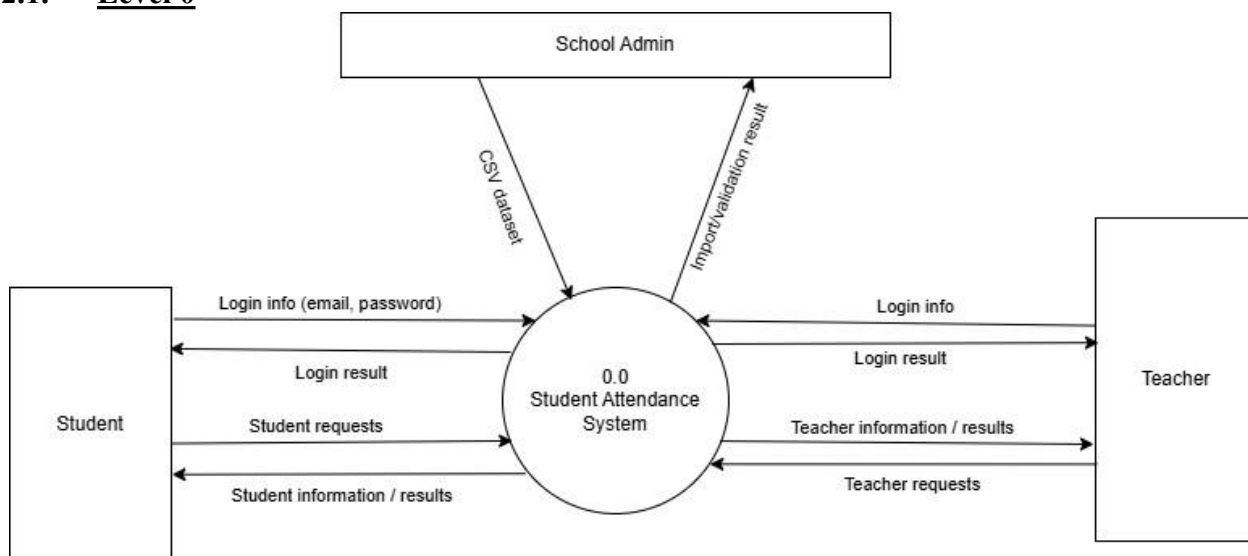
IV. Data Flow Diagram

1. FDD

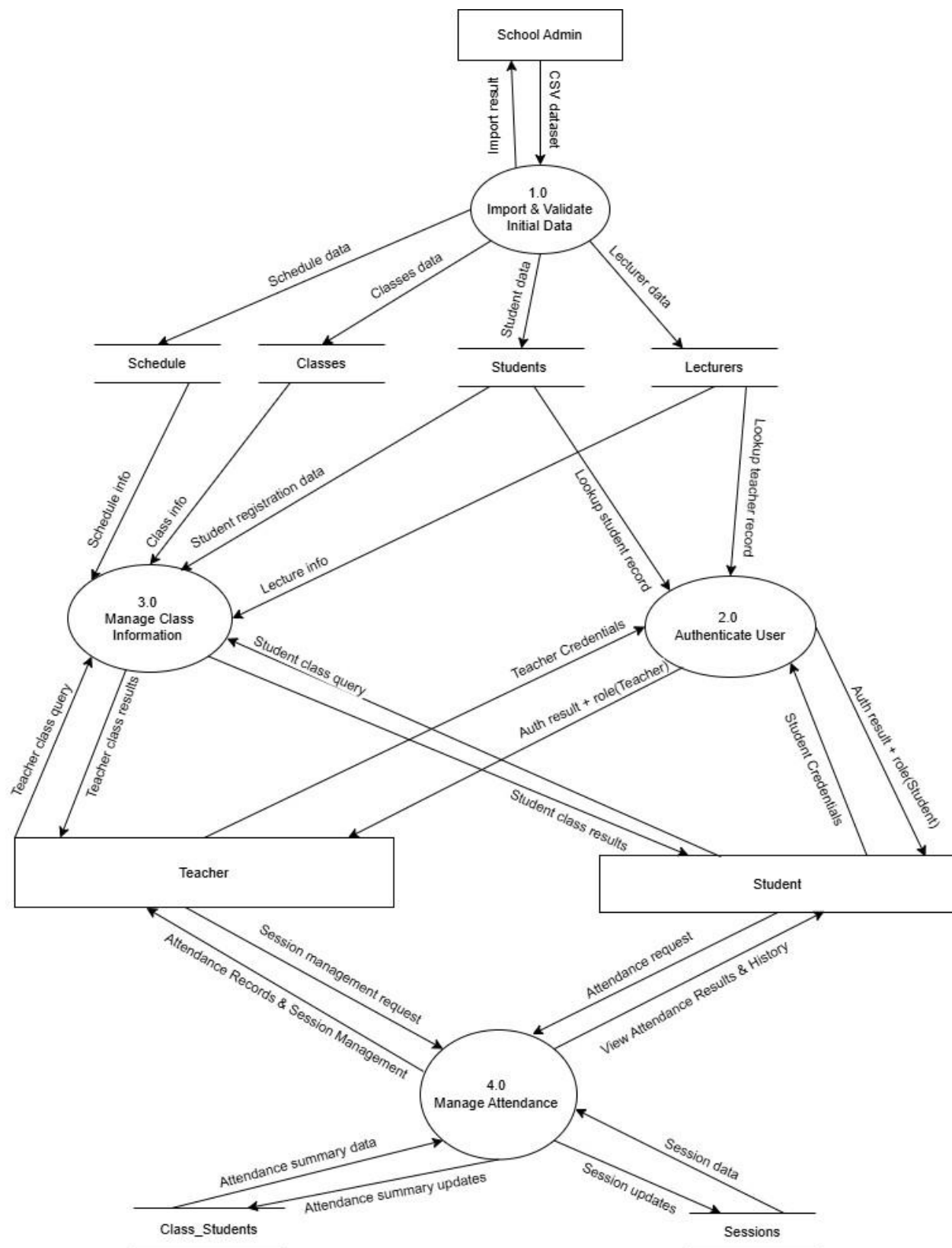


2. DFD

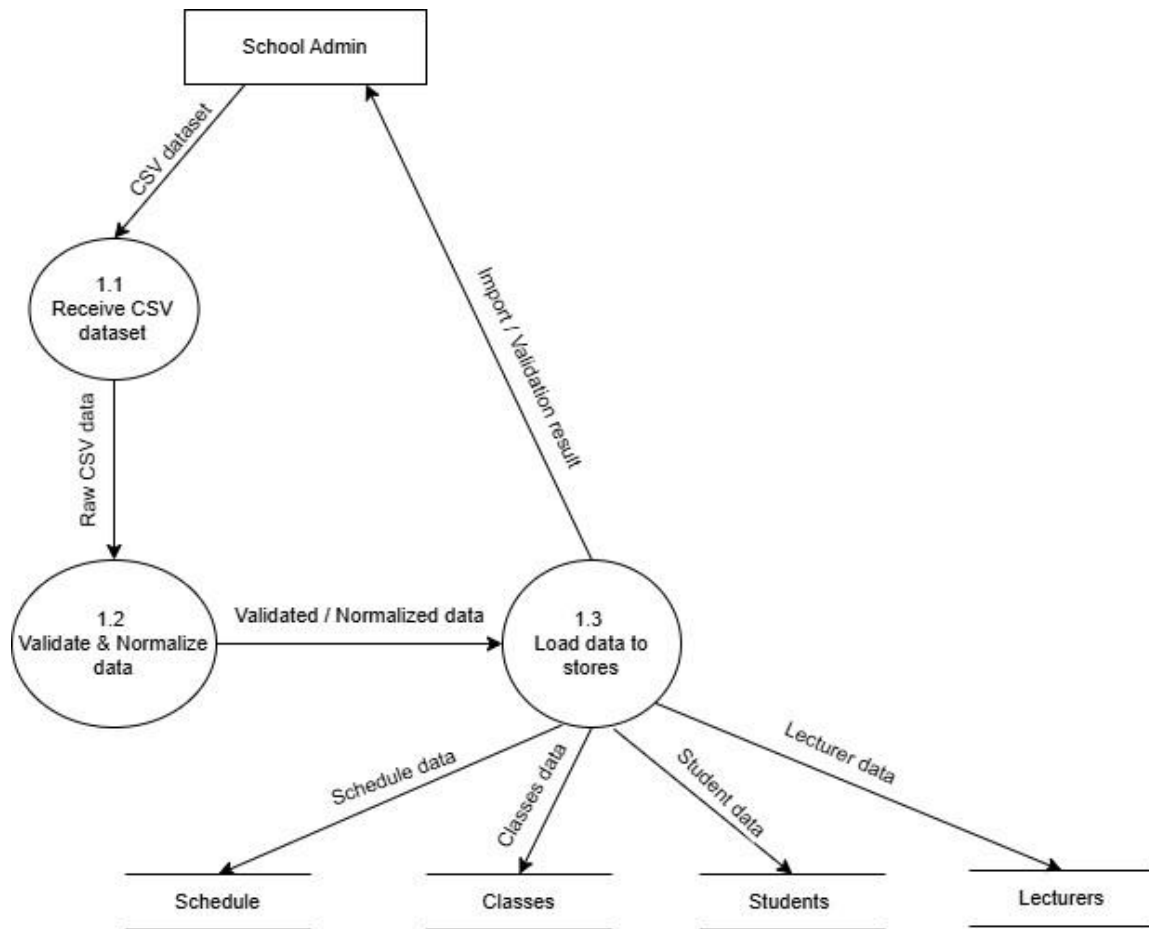
2.1. Level 0



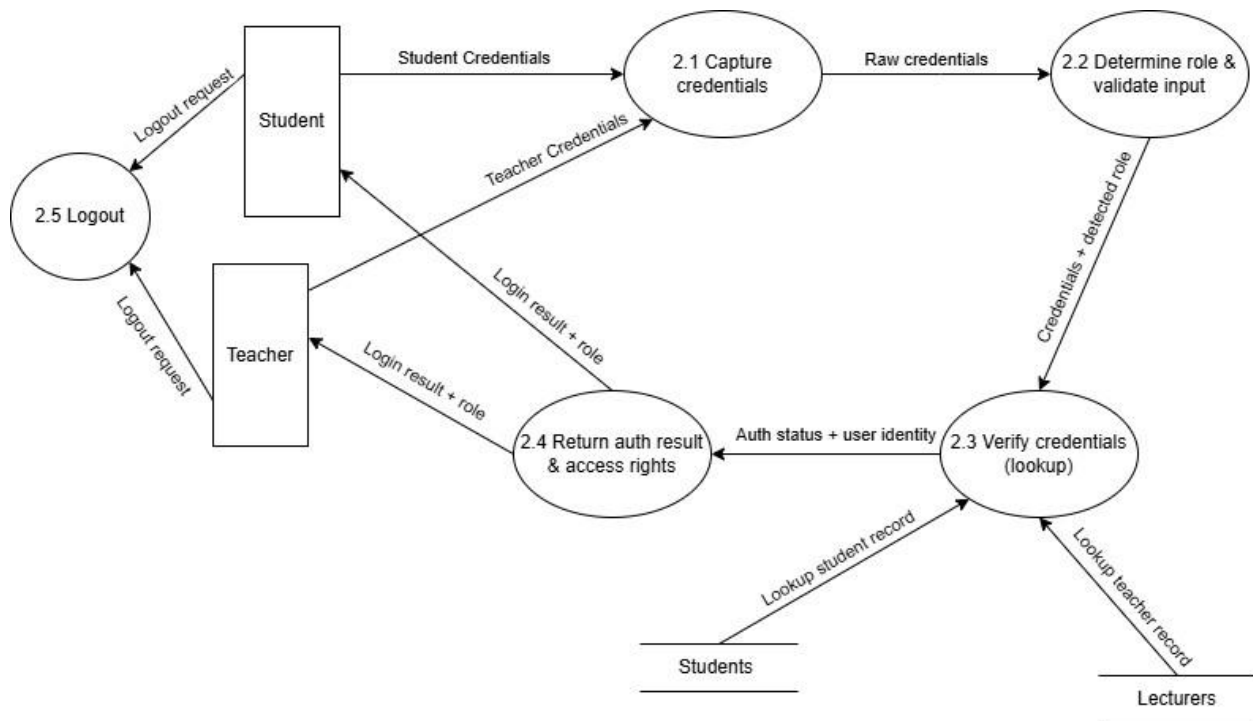
2.2. Level 1



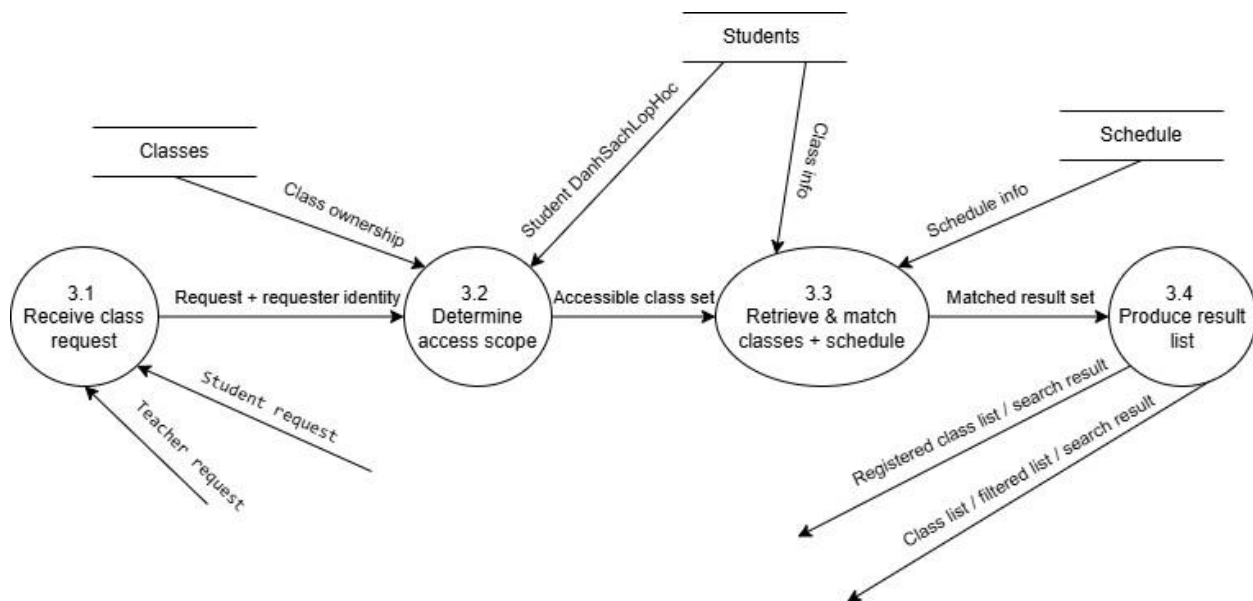
2.3. Level 2: 1.0 Import & Validate Initial Data



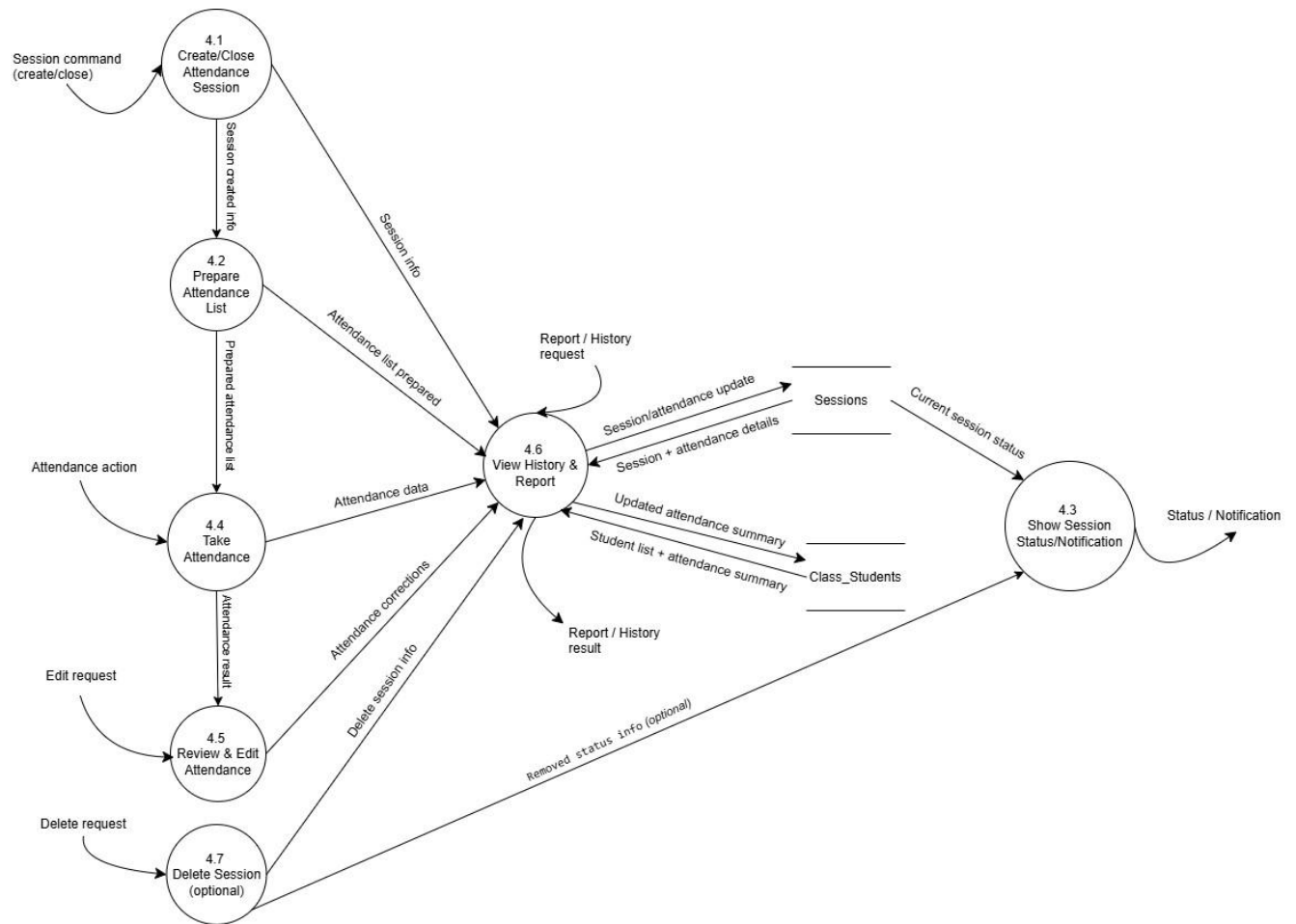
2.4. Level 2: 2.0 Authenticate User



2.5. Level 2: 3.0 Manage Class Information

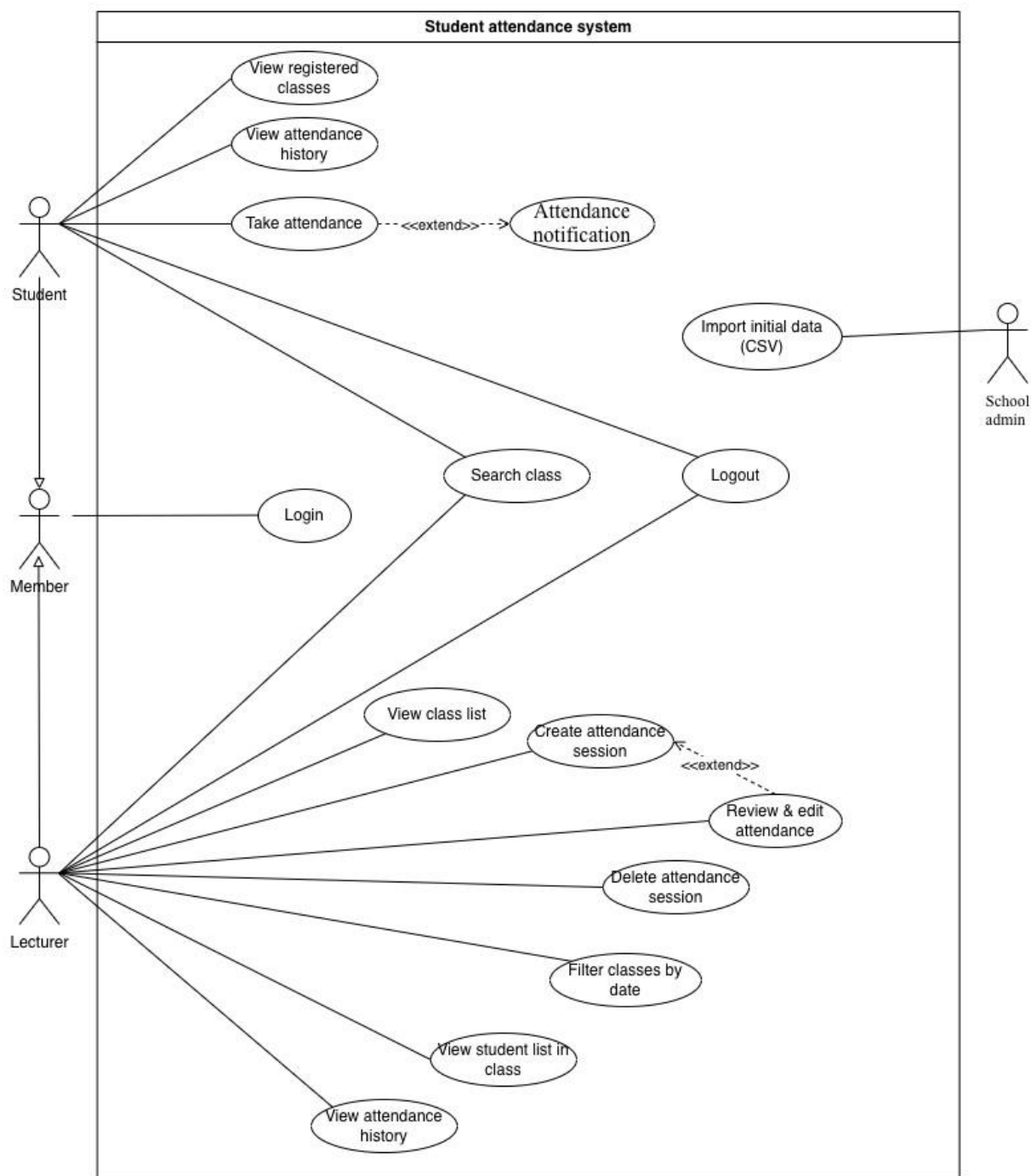


2.6. Level 2: 4.0 Manage Attendance



V. Use case Diagram

1. Use case diagram



1.1. List of Actors

No.	Actor	Meaning
1	Member	User who has not logged in yet (only access the Login screen).

No.	Actor	Meaning
2	Student	Student who logs in (account provided by the university).
3	Lecturer	Lecturer who logs in, views their teaching classes, searches classes, views student lists, etc. (account provided by the university).
4	School Admin	Provides the initial CSV data source for the system.

1.2. List of Use-cases

No.	Use case	Meaning
1	Import initial data (CSV)	Load data provided by the university for login, class management, and attendance.
2	Login	Student/Lecturer logs in using the university email @ut.edu.vn; Student uses student ID, Lecturer uses citizen ID; the system does not store a login session.
3	Logout	Exit the system/end the access session (because the system does not store a login session).
4	View class list	Lecturer views the list of classes they teach and the study time for a specific date (from class_schedule.csv, filtered by lecturer email).
5	Search class	Search a class by course name or class code; course name search is case-insensitive; class code search returns exactly one class.
6	Filter classes by date	Lecturer enters a date (DD/MM/YYYY) to view classes taught on that day (correct date + correct teaching permission).
7	View registered classes	Student views the list of registered classes after login (auto-shown based on student information).

8	Create attendance session	Lecturer creates an attendance session for a class: enter start/end time, can set a password; create a new row in the sessions file and leave svcomat empty.
No.	Use case	Meaning
9	Attendance notification	After creating a session, the system shows a notice in the class: the session is open and how much time is left; when expired, show “Attendance closed”.
10	Take attendance	Student takes attendance: (if required) enter the correct password; type “yes” to confirm; each student can take attendance only once per session; cannot take attendance after expired.
11	View student list in class	Lecturer views the student list of a class (name, student ID, attendance status).
12	Review & edit attendance	After the session is closed, the lecturer can edit manually (remove/add svcomat in class_[ClassCode]_sessions.csv) and update sobuoihoc.
13	View attendance history	Student views their own attendance history; Lecturer views history of sessions they created (open/closed, number of students attended by counting svcomat in class_[ClassCode]_sessions.csv).
14	Delete attendance session	Lecturer deletes an open/closed session; if deleting a closed session, subtract sobuoihoc for students who attended and update the CSV file again.

2. Use-case: Import initial data (CSV)

2.1. Summary

This use-case allows the system to load school-provided data (students, lecturers, classes, schedules) to support login, showing class list, and doing attendance.

2.2. Event Flow

2.2.1. Main flow

1. This use-case starts when the system boots up, or when system admin (or data department) provides the set of CSV files.
2. System reads student data (has school email, student code, list of classes).
3. System reads class data and schedule to determine which classes belong to which lecturer permission.
4. System initializes generated data for each class (like students/sessions) following the required structure. **2.2.2. Alternative flows** No alternative flow.

2.3. Special requirements

Class code / student code is unique, provided by school, and already preloaded.

2.4. Precondition (system state before) System has no data yet.

2.5. Postcondition (system state after)

Data is ready to do Login, view class, and do attendance.

2.6. Extension point

Use-case Login: only can be executed when data has been imported.

3. Use-case: Login

3.1. Summary

This use-case describes how Students/Lecturers login to the system using school email and password by the rule. **3.2. Event Flow**

3.2.1. Main flow

1. Use-case starts when user wants to access the system.
2. System requests user to input school email (@ut.edu.vn) and password.
3. User enters information and requests to login.
4. System validates the credentials with the preloaded data.
5. If correct, system shows message like “Access system successfully”.

3.2.2. Alternative flows

3.2.2.1. Wrong email/password

If user inputs wrong info, system shows: “Email or password is incorrect. Please try again.”

3.3. Special requirements

System does not store login session, so when accessing again user must login again.

3.4. Precondition

User is not logged in.

3.5. Postcondition

If success, user can access functions based on role (Student/Lecturer).

3.6. Extension point

Use-case View class list / View registered classes.

4. Use-case: Logout

4.1. Summary

This use-case allows user to exit the system / end the access.

4.2. Event Flow

4.2.1. Main flow

1. User chooses Logout/Exit.
2. System ends the access (because it does not keep session).

4.2.2. Alternative flows No alternative flow. **4.3. Special requirements**

None.

4.4. Precondition

User is currently accessing the system.

4.5. Postcondition

User exits the system.

4.6. Extension point

None.

5. Use-case: View class list

5.1. Summary

This use-case allows lecturer to view the list of teaching classes and study time by date, based on schedule and lecturer email.

5.2. Event Flow

5.2.1. Main flow

1. Lecturer logs in successfully.
2. Lecturer selects “view classes” function.
3. System filters class_schedule.csv and only shows classes where EmailGiangVien matches the lecturer email currently logged in.
4. System displays class info (subject name, class code, study date, start/end time, etc.).

5.2.2. Alternative flows None.

5.3. Special requirements

None.

5.4. Precondition

Lecturer already logged in.

5.5. Postcondition

Class list is displayed.

5.6. Extension point

- Use-case Search class
- Use-case Filter classes by date

6. Use-case: Search class

6.1. Summary

This use-case allows Student/Lecturer to search class by subject name or class code, but only inside the permission scope of the logged-in user.

6.2. Event Flow

6.2.1. Main flow

1. User is on the class list screen.
2. User inputs subject name or class code.
3. System searches:
 - + By subject name: case-insensitive, return list of matching classes.
 - + By class code: return exactly 1 class because code is unique.

4. System displays results within access permission scope. **6.2.2. Alternative flows**
None.

6.3. Special requirements

None.

6.4. Precondition

Student/Lecturer already logged in.

6.5. Postcondition

The searched class list is displayed.

6.6. Extension point

None.

7. Use-case Filter Classes by Date

7.1. Summary

This use case allows a lecturer to filter the list of classes they are teaching by a specific date in order to view classes scheduled on that day.

The filtering result includes only the classes taught by the currently logged-in lecturer.

7.2. Flow of Events

7.2.1. Main Flow

The lecturer has successfully logged into the system.

The lecturer is on the View class list screen.

The lecturer enters the desired date in DD/MM/YYYY format.

The lecturer selects the Filter classes by date function.

The system checks the class schedule data stored in the class_schedule.csv file.

The system filters and displays the classes that:

Have a scheduled date matching the entered date

Have a lecturer email matching the email of the currently logged-in lecturer

The system displays the list of classes that meet the conditions (course name, class ID, class time, etc.).

7.2.2. Alternative Flows

7.2.2.1. No classes on the selected date

If no classes satisfy the filtering conditions, the system displays the message: “No classes are scheduled on the selected date.”

7.2.2.2. Invalid date format

If the lecturer enters an invalid date format, the system displays the message: “Invalid date format. Please enter the date in DD/MM/YYYY format.”

7.3. Special Requirements

The entered date must follow the DD/MM/YYYY format.

The lecturer can only view classes they are responsible for and cannot view classes taught by other lecturers.

The filtering operation does not modify the original data stored in the CSV files.

7.4. Pre-conditions

The lecturer has logged into the system.

Class and schedule data have been loaded from CSV files.

The system is currently displaying the lecturer’s class list.

7.5. Post-conditions

The class list is displayed based on the selected date.

No changes are made to the stored data.

7.6. Extension Points

Use-case View class list (this use case is performed on the displayed class list).

8. Use-case View Registered Classes

8.1. Summary

This use case allows a student to view the list of classes they have registered for after logging into the system.

The class list is automatically filtered based on the student’s ID.

8.2. Flow of Events

8.2.1. Main Flow

The student logs into the system successfully.

The student selects the View registered classes function.

The system retrieves the student's information from the loaded data.

The system determines the classes that the student has registered for.

The system displays the class list (course name, class ID, schedule).

8.2.2. Alternative Flows

8.2.2.1. Student has not registered for any classes

The system displays the message: "You have not registered for any classes."

8.3. Special Requirements

The student can only view their own registered classes.

Data modification is not allowed.

8.4. Pre-conditions The student has logged in.

Class data has been loaded.

8.5. Post-conditions

The registered class list is displayed.

8.6. Extension Points

Use-case Take attendance

Use-case View attendance history

9. Use-case Create Attendance Session

9.1. Summary

This use case allows a lecturer to create an attendance session for a class, including the start time, end time, and an optional attendance password.

9.2. Flow of Events

9.2.1. Main Flow

The lecturer logs into the system successfully.

The lecturer selects a class they are teaching.

The lecturer selects the Create attendance session function.

The lecturer enters:

Start time

End time

Password (if any)

The system creates a new attendance session.

The system stores the session information in the sessions.csv file.

The system initializes the list of present students (svcomat) as empty.

9.2.2. Alternative Flows

9.2.2.1. Invalid time

If the end time is earlier than the start time, the system displays an error message.

9.3. Special Requirements

Each class can have only one active attendance session at a time.

The lecturer can only create sessions for classes they teach.

9.4. Pre-conditions

The lecturer has logged in.

The class exists and is under the lecturer's responsibility.

9.5. Post-conditions

A new attendance session is created and stored.

9.6. Extension Points

Use-case Attendance notification

10. Use-case Attendance Notification

10.1. Summary

This use case allows the system to display notifications about the status of an attendance session, including the remaining time or the ended status.

10.2. Flow of Events

10.2.1. Main Flow

An attendance session is successfully created.

The system displays notifications:

Attendance session is active

Remaining time

When the session expires, the system updates the notification to: “Attendance session has ended.”

10.3. Special Requirements

Notifications are updated automatically in real time.

Students cannot interact with the notifications.

10.4. Pre-conditions

An attendance session has been created.

10.5. Post-conditions

The attendance session status is displayed correctly.

10.6. Extension Points

Use-case Create attendance session

11. Use-case Take Attendance

11.1. Summary

This use case allows a student to confirm their attendance in an active attendance session.

11.2. Flow of Events

11.2.1. Main Flow

The student logs into the system successfully.

The student accesses a class with an active attendance session.

If a password is required, the student enters the password.

The student enters “present” to confirm attendance.

The system records the student as present.

The system updates:

svcomat sobuoihoc

11.2.2. Alternative Flows Incorrect password

Duplicate attendance attempt

Attendance session has expired

(The system displays corresponding messages.)

11.3. Special Requirements

Each student can take attendance only once per session.

11.4. Pre-conditions The student has logged in.

An attendance session is active.

11.5. Post-conditions

Attendance status is updated.

11.6. Extension Points

Use-case Review & edit attendance

Use-case View attendance history

12. Use-case View Student List in Class

12.1. Summary

This use case allows a lecturer to view the list of students in a class, including their attendance status.

12.2. Flow of Events

12.2.1. Main Flow

The lecturer logs into the system.

The lecturer selects a class.

The lecturer selects View student list in class.

The system displays the student list and attendance status.

12.3. Special Requirements

Only the assigned lecturer is allowed to view the student list.

12.4. Pre-conditions The class exists.

12.5. Post-conditions

The student list is displayed.

12.6. Extension Points

Use-case Review & edit attendance

13. Use-case Review & Edit Attendance

13.1. Summary

This use case allows a lecturer to edit students' attendance status after an attendance session has ended.

13.2. Flow of Events

13.2.1. Main Flow

The lecturer selects a completed attendance session.

The system displays the student list.

The lecturer edits the status (Present/Absent).

The system updates the data.

13.3. Special Requirements

Editing is allowed only after the session has ended.

13.4. Pre-conditions

The attendance session has ended.

13.5. Post-conditions

Attendance data is updated.

13.6. Extension Points

Use-case Delete attendance session

14. Use-case View Attendance History

14.1. Summary

This use case allows students and lecturers to view attendance history.

14.2. Flow of Events

14.2.1. Main Flow

The user logs into the system.

The user selects View attendance history.

The system displays attendance history based on the user's role.

14.3. Special Requirements

Students can only view their own attendance history.

Lecturers can only view sessions they have created.

14.4. Pre-conditions

The user has logged in.

14.5. Post-conditions

Attendance history is displayed.

14.6. Extension Points

None.

15. Use-case Delete Attendance Session

15.1. Summary

This use case allows a lecturer to delete an attendance session, including completed sessions.

15.2. Flow of Events

15.2.1. Main Flow

The lecturer selects an attendance session.

The lecturer selects Delete attendance session.

The system deletes the session.

If the session has already ended: The system updates sobuoihoc.

15.3. Special Requirements

Deleted sessions cannot be restored.

15.4. Pre-conditions

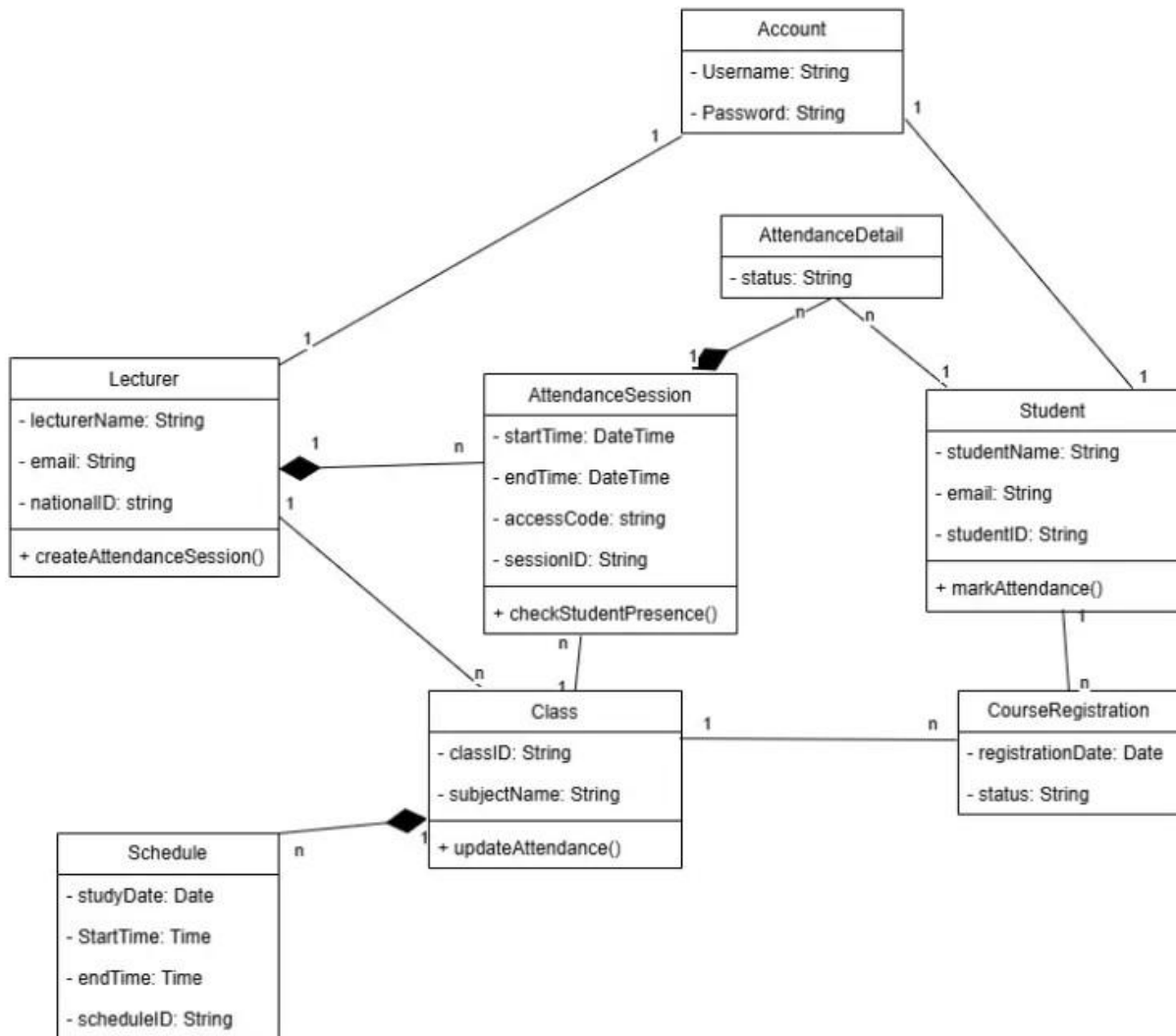
The attendance session exists.

15.5. Post-conditions

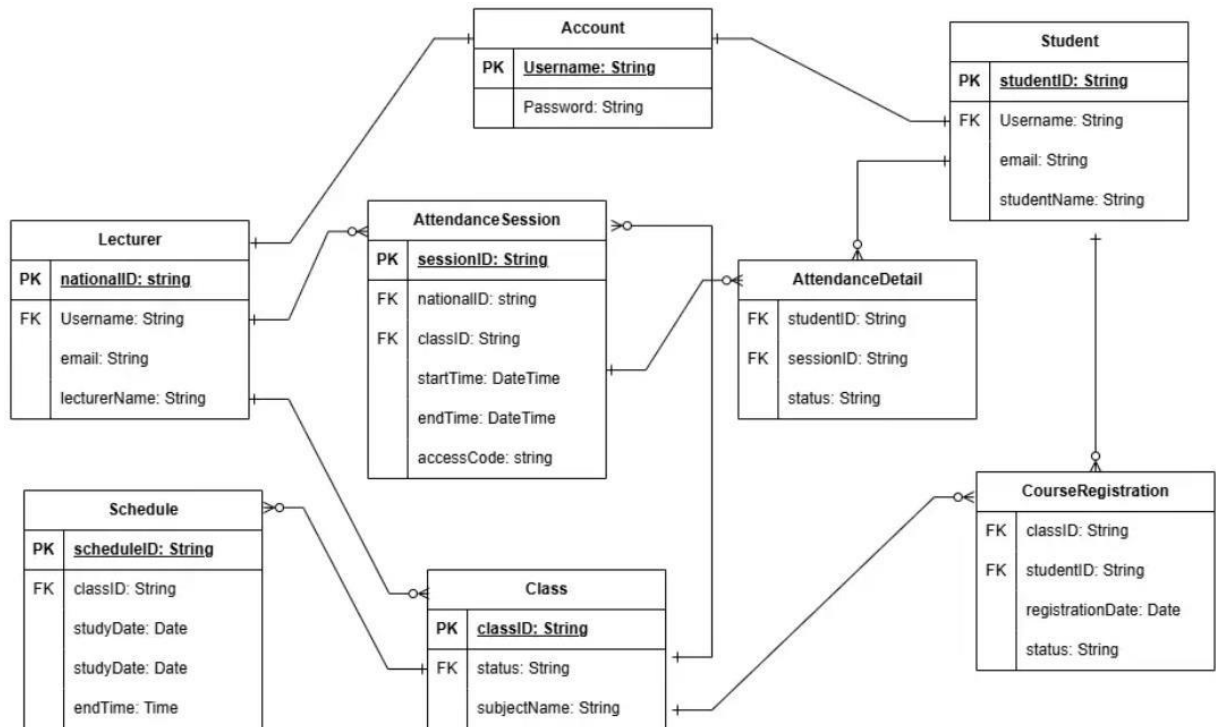
The attendance session is removed from the system.

15.6. Extension Points None.

VI. Class Diagram



VII. Data Model



VIII. Interface Design Description

The student attendance system uses a menu-driven interface with a loop-menu structure. After each operation, the system returns to the corresponding menu until the user selects logout to exit the system.

The system does not store login sessions; therefore, users must log in again each time they access the system.

Interface screens

1. Login

```

.....
                        STUDENT ATTENDANCE SYSTEM
                        .....

1. Login
0 . Exit

Selection: _
  
```

- Login:

- The system requires users to enter their institutional email (ending with @ut.edu.vn) and password.
- Students use their student ID as the password.
- Lecturers use their national ID number (CCCD) as the password.
- If the login information is incorrect, the system displays the message: “Incorrect email or password. Please try again.”
- If the login is successful, the system redirects the user to the appropriate interface based on their role (Student or Lecturer).

- Exit:

- Terminates the program and closes the system.

2. Lecturer dashboard

LECTURER DASHBOARD

-
1. View list of assigned classes
 2. Search / Filter classes
 3. View student list in a class
 4. Create an attendance session
 5. Review and edit attendance
 6. View attendance history
 7. Delete an attendance session
 0. Logout
-

Selection: _

- View list of assigned classes:

- Displays the classes the lecturer is responsible for, along with class dates and time slots retrieved from the schedule data.

- Search/Filter classes:

- Allows the lecturer to search for classes by course name or class code, or filter classes by a specific date.

- View student list in a class:
- Displays the list of students enrolled in the selected class along with their attendance status for the current session.
- Create an attendance session:
- Allows the lecturer to create a new attendance session by entering the start time, end time, and an optional password.
- Review and edit attendance:
- After an attendance session is closed, the lecturer can review the student list and manually edit attendance statuses if necessary.
- View attendance history:
- Displays all attendance sessions created for each class, including session status and the number of students who attended.
- Delete an attendance session:
- Allows the lecturer to delete any attendance session after confirmation and updates the related attendance data accordingly.
- Logout:
- Ends the session and returns to the login screen.

3. Student dashboard

STUDENT DASHBOARD

-
1. View registered classes
 2. Search classes
 3. Mark attendance
 4. View attendance history
 0. Logout
-

Selection: _

- View registered classes:
 - Displays the classes the student has registered for, based on preloaded system data.
- Search classes:
 - Allows the student to search for classes by course name or class code within the classes they are authorized to access.
- Mark attendance:
 - When an attendance session is open, the student can mark attendance by entering the password (if required) and confirming their presence.
- View attendance history:
 - Displays the student's attendance history for each class, including "Present/Absent" status and the total number of attended sessions.
- Logout:
 - Ends the session and returns to the login screen.